

Задание 3

Часть 1

Используя LLM (например, `deerpseek`) и дополнительную техническую литературу, справочные сайты, выполнить анализ программного и информационного обеспечения сайта кафедры используя ГОСТ 19.503-79, составить фрагмент руководства системного программиста, где описать раздел 2 этого руководства.

1. Общие сведения о программе

Настоящий документ описывает структуру, состав и особенности программного обеспечения информационного сайта кафедры информационных и коммуникационных технологий (ИКТ) РГПУ им. А. И. Герцена, функционирующего по адресу **ict.herzen.spb.ru**.

Система представляет собой веб-приложение, разработанное на базе CMS Grav, развёрнутое в контейнерной инфраструктуре Kubernetes. Приложение обеспечивает публикацию учебных материалов, расписания и новостей кафедры, а также предоставляет интеграцию с сервисами Google (Google Calendar, Google Drive).

Технические и программные средства, обеспечивающие выполнение программы:

- Операционная система: Linux (Ubuntu) в Docker-контейнере
 - Веб-сервер: Nginx
 - Язык серверной логики: PHP 7.x
 - CMS: Grav (плоскофайловая система управления контентом)
 - Среда выполнения контейнеров: Docker
 - Оркестрация контейнеров: Kubernetes (namespace: default)
 - CI/CD: GitLab CI/CD (Kaniko для сборки образов)
 - Реестр контейнеров: registry-git.herzen.spb.ru
 - Внешние API: Google OAuth 2.0, Google Calendar API, Google Drive API
-

2. Структура программы

2.1. Общая архитектура

Программа имеет клиент-серверную архитектуру. Пользователь взаимодействует с системой через веб-браузер по протоколу HTTP/HTTPS. Запросы поступают через Ingress-контроллер Kubernetes на Service **apps-ict**, который направляет трафик к Pod'у с работающим контейнером приложения.

Схема взаимодействия компонентов:

```
Браузер пользователя
    ↓
Ingress (nginx)
host: ict.herzen.spb.ru
```

```

↓
Service apps-ict (port 80)
↓
Pod (контейнер Grav)
Nginx + PHP-FPM + файлы CMS
↓
Google API (OAuth 2.0 / Calendar / Drive)

```

2.2. Состав программы

Компонент	Файл / Ресурс	Описание
CMS Grav	grav/user/	Контент, темы и плагины сайта кафедры
Docker-образ	Dockerfile	Образ на базе dmitriizolotov2/grav ; копирует пользовательские данные
Конфигурация Kubernetes	deploy.yaml	Описание Service, Ingress и Deployment для кластера
CI/CD-конвейер	.gitlab-ci.yml	Три стадии: сборка образа, публикация в кластер, очистка реестра
Конфигурация веб-сервера	.htaccess	Правила mod_rewrite: маршрутизация, блокировка эксплойтов, защита каталогов
Google OAuth 2.0	oauth2callback.php	Получение кода авторизации и токена доступа к Google API
Модуль Google Drive	pr_gd_save_callback.php	Загрузка файлов из Google Drive; конвертация Google Docs в DOCX/XLSX/PDF
Модуль настройки GCA	pr_gca_save_callback.php	Сохранение соответствия страниц Grav и почтовых адресов в gca.json
Модуль настройки GDA	pr_gda_save_callback.php	Сохранение соответствия страниц Grav и ID папок Google Drive в gda.json
Учётные данные Google API	client_id.json	OAuth 2.0 client_id и client_secret для проекта spb-rgpu-ict-site-2018

2.3. Связи между составными частями

- **Dockerfile** использует базовый образ [dmitriizolotov2/grav](#) и копирует в него содержимое каталога [grav/user/](#), PHP-скрипты и [client_id.json](#).
- **GitLab CI** ([.gitlab-ci.yml](#)) автоматически собирает Docker-образ посредством Kaniko и публикует его в реестр при появлении новых тегов или коммитов в ветку [master](#).
- После сборки pipeline применяет [deploy.yaml](#) к кластеру Kubernetes, подставляя актуальный тег образа.
- **deploy.yaml** описывает три объекта Kubernetes: Service (маршрутизация трафика на порт 80), Ingress (публикация на хосте [ict.herzen.spb.ru](#)) и Deployment (один экземпляр Pod с

контейнером приложения).

- **oauth2callback.php** при первичной авторизации получает токен Google OAuth 2.0 и сохраняет его для последующего использования скриптами **pr_gd_save_callback.php** и **pr_gca_save_callback.php**.
- **pr_gd_save_callback.php** читает токен из **json.txt**, при истечении срока обновляет его через **refresh.txt**, обращается к Google Drive API и возвращает файл пользователю.
- **pr_gca_save_callback.php** и **pr_gda_save_callback.php** сохраняют конфигурационные JSON-файлы (**gca.json**, **gda.json**), которые используются плагинами Grav для отображения данных Google Calendar и Google Drive на страницах сайта.
- Файл **.htaccess** обеспечивает маршрутизацию всех запросов через **index.php** CMS Grav, а также защиту системных каталогов и блокировку потенциально опасных запросов.

2.4. Связи с другими программами

Внешняя система	Протокол / API	Назначение
Google OAuth 2.0	HTTPS, OAuth 2.0	Аутентификация и авторизация доступа к сервисам Google
Google Calendar API	HTTPS, REST	Получение данных о событиях и расписании для отображения на сайте
Google Drive API	HTTPS, REST	Получение и скачивание файлов, хранящихся на Google Drive
GitLab Registry	HTTPS, Docker Registry API v2	Хранение Docker-образов приложения
Kubernetes API	HTTPS, kubectl	Управление развёртыванием приложения в кластере
GitLab CI	GitLab Runner + Kaniko	Автоматическая сборка и публикация образов

3. Настройка программы

Настройка программы выполняется на следующих уровнях:

- **Переменные окружения GitLab CI:** **CI_REGISTRY**, **CI_REGISTRY_USER**, **CI_REGISTRY_PASSWORD** — задаются в настройках проекта GitLab и используются для аутентификации при сборке и публикации образа.
- **Конфигурация Kubernetes:** файл **/config-production** (KUBECONFIG) должен быть доступен на runner-узле для выполнения команд **kubectl**. Количество реплик (по умолчанию 1) задаётся в **deploy.yaml**.
- **Учётные данные Google API:** файл **client_id.json** должен быть размещён в каталоге **php_scripts/** внутри контейнера. Redirect URI должен совпадать с адресом сервера: **http://<host>/oauth2callback.php**.
- **Настройка CMS Grav:** конфигурационные файлы размещены в **grav/user/config/**. Активная тема задаётся в **system.yaml**.
- **Соответствие страниц и данных Google:** через административный интерфейс Grav данные сохраняются в **gca.json** (страница → email) и **gda.json** (страница → ID папки Google Drive).

4. Проверка программы

Для проверки работоспособности программы используются следующие методы:

- **Проверка сборки образа:** выполнить коммит в ветку `master`; стадия `build` в GitLab CI должна завершиться без ошибок.
- **Проверка развёртывания:** после успешной сборки стадия `publish` применяет `deploy.yaml`; команда `kubectl get pods -n default` должна показывать Pod в статусе `Running`.
- **Проверка доступности сайта:** обратиться по адресу `http://ict.herzen.spb.ru/` и убедиться в отображении главной страницы.
- **Проверка Google OAuth:** перейти по адресу `/oauth2callback.php`; при корректной настройке произойдёт редирект на страницу авторизации Google.
- **Проверка Google Drive:** запросить файл через `/pr_gd_save_callback.php` с параметрами `id`, `type`, `filename`; браузер должен начать загрузку файла.

5. Сообщения системному программисту

Сообщение / Ситуация	Причина	Действие
Pod в статусе <code>CrashLoopBackOff</code>	Ошибка в PHP-коде или отсутствие <code>client_id.json</code>	Проверить логи: <code>kubectl logs <pod-name></code> ; убедиться, что Dockerfile корректно копирует все файлы
<code>ImagePullBackOff</code>	Неверные учётные данные реестра или образ не существует	Проверить секрет: <code>kubectl get secret regsecret</code> ; пересоздать при необходимости
HTTP 403 при обращении к файлам	Правила <code>.htaccess</code> блокируют запрос	Проверить правила RewriteRule; убедиться, что <code>mod_rewrite</code> включён
Google API: <code>invalid_client</code>	Неверный <code>client_id</code> или <code>client_secret</code>	Пересоздать OAuth-клиент в Google Cloud Console и обновить <code>client_id.json</code>
Google API: <code>Token has been expired</code>	Истёк <code>refresh_token</code> или файл <code>refresh.txt</code> повреждён	Удалить <code>json.txt</code> и <code>refresh.txt</code> ; повторно пройти авторизацию через <code>/oauth2callback.php</code>
Очистка реестра не выполняется (стадия <code>cleanup</code>)	Устаревшие учётные данные в <code>.gitlab-ci.yml</code>	Обновить пароль пользователя <code>cleaner</code> в секции <code>cleanup</code>

Часть 2

Составить список используемого программного обеспечения, задействованных технологий, указав для чего соответствующая технология или ПО используется.

Список используемого программного обеспечения и технологий

Технология / ПО	Версия	Назначение
Grav CMS	актуальная (образ dmitriizolotov2/grav)	Плоскофайловая CMS для управления контентом сайта кафедры
Nginx	входит в базовый Docker-образ	Веб-сервер; обработка HTTP-запросов и проксирование на PHP-FPM
PHP	7.x	Серверная логика: интеграция с Google API, обработка форм
Google API PHP Client	входит в vendor/ Grav	Библиотека для работы с Google OAuth 2.0, Calendar API, Drive API
Docker	актуальная	Контейнеризация приложения; изоляция зависимостей
Kubernetes	extensions/v1beta1 / apps/v1	Оркестрация контейнеров; управление масштабированием и доступностью
GitLab CI/CD	актуальная	Автоматизация сборки, тестирования и развёртывания
Kaniko	gcr.io/kaniko-project/executor:debug	Сборка Docker-образов внутри Kubernetes без Docker daemon
Python 3.6	python:3.6-alpine (для cleanup)	Запуск скрипта очистки реестра GitLab (gricleaner.py)
Apache mod_rewrite	входит в конфигурацию сервера	URL-маршрутизация, защита от эксплойтов (.htaccess)
Google OAuth 2.0	v2	Аутентификация пользователей и авторизация доступа к Google-сервисам
Google Calendar API	v3	Отображение расписания и событий кафедры на страницах сайта
Google Drive API	v3	Предоставление доступа к документам кафедры на Google Drive